图像颜色直方图

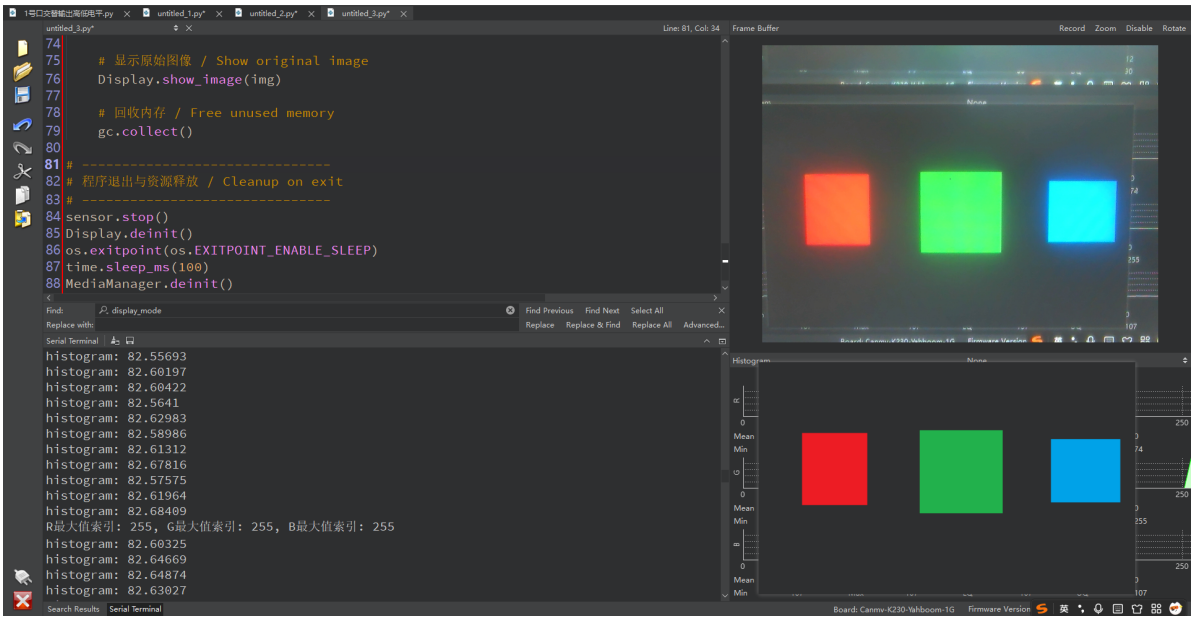
图像颜色直方图

- 例程效果
- 原理讲解
  - 什么是图像直方图
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 图像处理与直方图计算
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/19.rgb888_calc_histogram.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像直方图计算。



原理讲解

什么是图像直方图

图像直方图（Histogram）是一种用于表示图像像素强度分布的统计工具。它通过统计图像中每个像素值的出现频率来描述图像的亮度、对比度和颜色分布。对于彩色图像如 RGB888，直方图通常针对每个颜色通道（Red、Green、Blue）分别计算。

- **基本概念**：
  - 像素值范围：对于8位图像，像素值从0（黑色）到255（白色）。
  - 直方图表示：一个数组或图表，其中x轴表示像素值（0~255），y轴表示该像素值的出现频率。

    - 对于RGB888图像，直方图是一个3通道数组（每个通道256个bins），分别统计R、G、B通道的像素分布。
- **计算过程**：
    - 遍历图像像素，统计每个像素值在每个通道的计数。
    - 结果是一个形状为3x256的数组（例如，hist[0]为B通道，hist[1]为G通道，hist[2]为R通道）。
- **应用**：
    - **分析图像特性**：帮助评估图像的亮度（直方图偏向左侧表示暗，偏向右侧表示亮）、对比度（分布均匀表示高对比度）。
    - **图像增强**：基于直方图进行直方图均衡化，以改善图像质量。
    - **在该代码中**：计算直方图后，提取每个通道的最大值索引（即像素值频率最高的bin），用于监测图像的颜色分布或异常。

直方图是图像处理的基础工具，常用于预处理阶段，如自动曝光调整或目标检测。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *      # 摄像头接口 / Camera interface
from media.display import *     # 显示接口 / Display interface
from media.media import *       # 媒体资源管理器 / Media manager
import _thread
import cv_lite                  # cv_lite 扩展模块，包含 histogram 函数 / C extension
module for image processing
import ulab.numpy as np         # ulab 数组模块 / NumPy-like ndarray for
MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的直方图计算）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=100)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=100` 设置图像传输质量为最高。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

图像处理与直方图计算

```
clock = time.clock()
count = 0

while True:
    count += 1
    clock.tick()  # 记录当前帧开始时间 / Start timing

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray
reference (HWC)

    # 使用 cv_lite 计算 RGB 直方图（返回 shape 为 3x256 的数组）
    # Calculate RGB888 histogram using cv_lite (3x256 array)
    hist = cv_lite.rgb888_calc_histogram(image_shape, img_np)

    # 每 30 帧打印一次最大值索引 / Print max bin index every 30 frames
    if count == 30:
        r_hist = hist[2]  # R 通道 / Red channel
        g_hist = hist[1]  # G 通道 / Green channel
        b_hist = hist[0]  # B 通道 / Blue channel
        r_max = int(np.argmax(r_hist))  # R 最大值索引 / Max index of red channel
        g_max = int(np.argmax(g_hist))  # G 最大值索引 / Max index of green
channel
        b_max = int(np.argmax(b_hist))  # B 最大值索引 / Max index of blue channel
        print(f"R最大值索引: {r_max}, G最大值索引: {g_max}, B最大值索引: {b_max}")
        count = 0

    # 打印当前帧率 / Print FPS
    print("histogram:", clock.fps())

    # 显示原始图像 / Show original image
    Display.show_image(img)

    # 回收内存 / Free unused memory
    gc.collect()
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **直方图计算**：调用 `cv_lite.rgb888_calc_histogram()` 函数计算图像的RGB直方图，返回一个形状为3x256的数组（每个通道的256个bins）。
- **数据分析**：每30帧，使用 `np.argmax()` 找到每个通道直方图的最大值索引（即频率最高的像素值），并打印输出。
- **显示和输出**：显示原始图像，打印当前帧率，并进行垃圾回收以管理内存。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

本代码中没有显式可调参数（如阈值或内核大小）。如果需要优化性能，可以在主循环中调整计数器（如`count == 30`的间隔）来控制打印频率，但这不直接影响直方图计算。